LAB 3- Detecting Anomalies with Prophet

Confirming the app.py for app_one is still the same.

app.py U

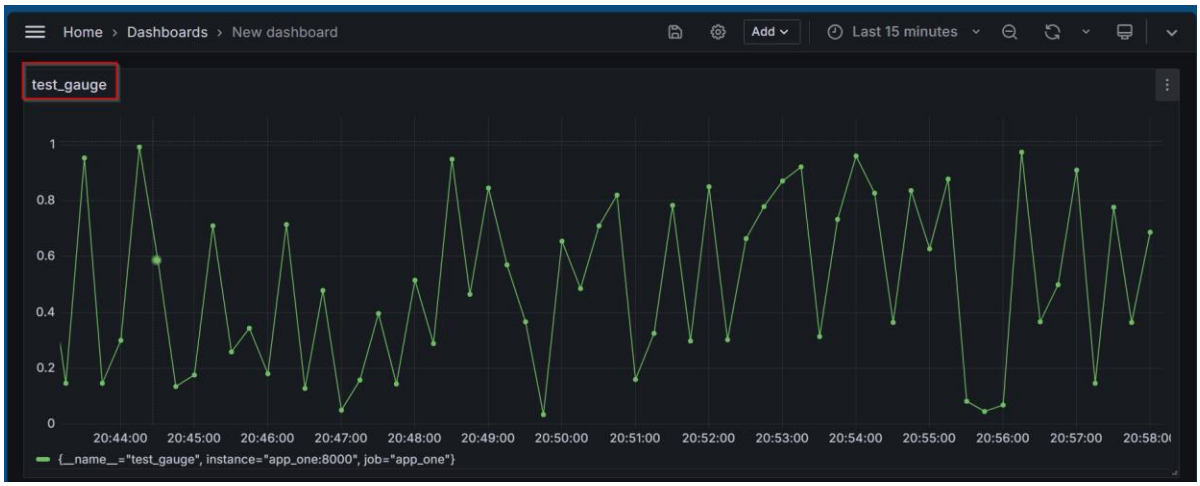
LAB3-Detecting_Anomalies_with_Prophet > containers > app_one > app.py

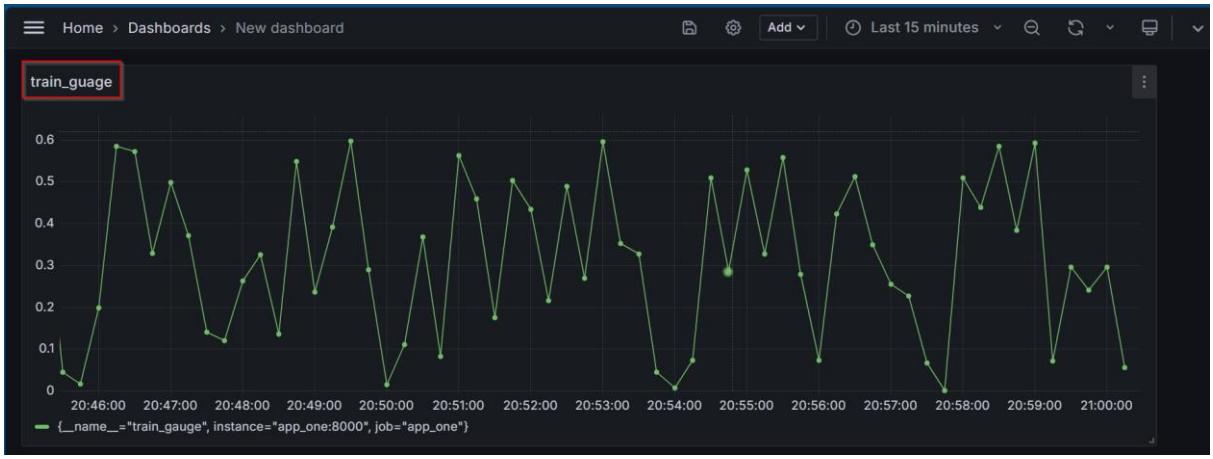
```python
from prometheus_client import start_http_server, Gauge, Histogram
import random
import time

# Define metrics
test_gauge = Gauge('test_gauge', 'Test Gauge between 0 and 1')
train_gauge = Gauge('train_gauge', 'Train Gauge between 0 and 0.6')
test_hist = Histogram('test_hist', 'Test Histogram between 0 and 1', buckets=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0))
train_hist = Histogram('train_hist', 'Train Histogram between 0 and 0.6', buckets=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6))

def emit_data():
    """Emit fake data"""
    time.sleep(5)
    test_value = random.uniform(0, 1)
    train_value = random.uniform(0, 0.6)
    test_gauge.set(test_value)
    train_gauge.set(train_value)

    test_hist.observe(test_value)
    train_hist.observe(train_value)



if __name__ == '__main__':
    start_http_server(8000)
    while True:
        emit_data()
```

Revisited the app _one from Lab 2 and created the containers with docker compose up -d

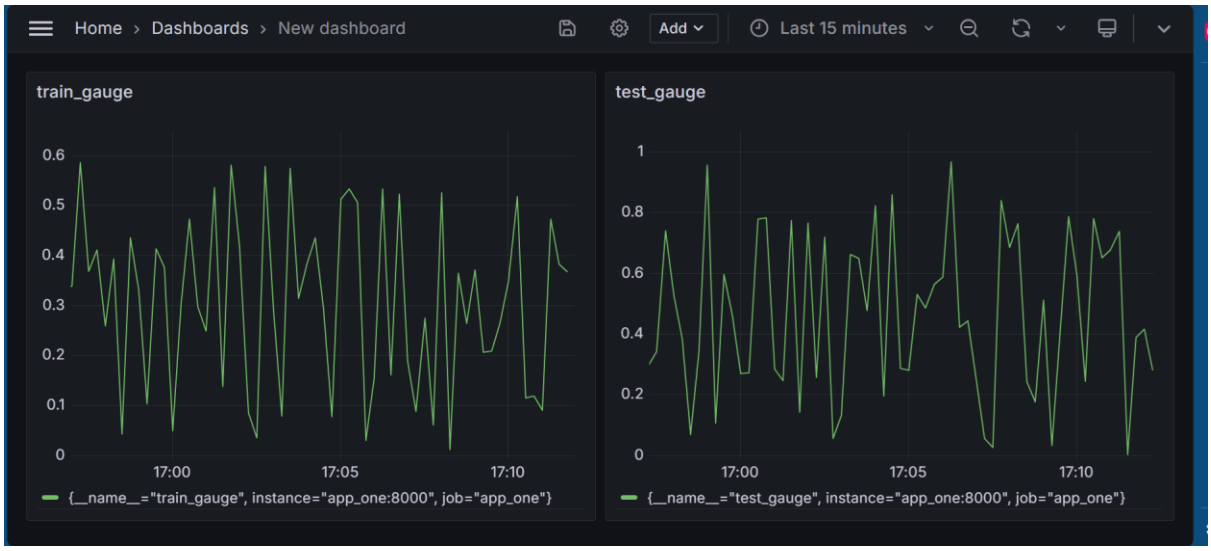
```
oadewusi@oluSec:~/aiops/Assignments/LAB3-Detecting_Anomalies_with_Prophet$ docker compose up -d
[+] Running 7/7
 ✓ Container prometheus_node_exporter  Started
 ✓ Container prometheus                Started
 ✓ Container app_one                   Started
 ✓ Container prometheus_push_gateway   Started
 ✓ Container postgres                  Started
 ✓ Container postgres_exporter         Started
 ✓ Container grafana                   Started
```

Screenshots on Grafana for the **test_guage** and **train_guage** metrics seperately

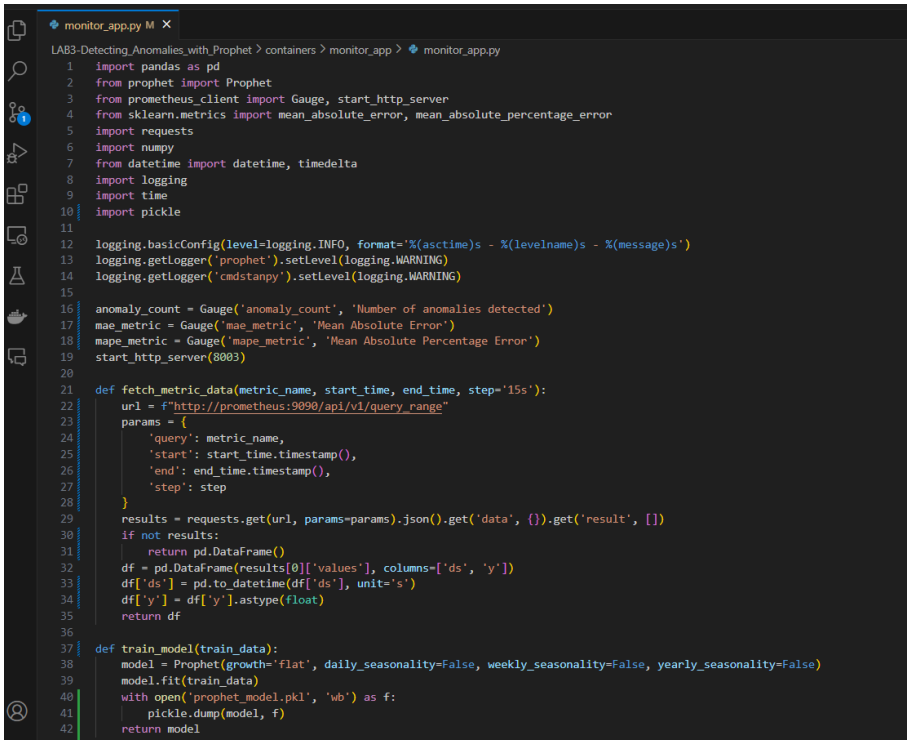




The metrics together on the dashboard:



Prepared a python application called **monitor_app** that reads the training data metric for a period of time, builds a Prophet model from it, and evaluates the model against the test metric. Simply print the anomalies detected to the console log.



Now packaging the application as a docker image:
Created a Dockerfile:

Dockerfile U

LAB3-Detecting_Anomalies_with_Prophet > containers > monitor_app > Dockerfile > ...

```
FROM python:3.9-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8003
ENV PYTHONBUFFERED=1
CMD ["python", "monitor_app.py"]
```

With the requirements:

requirements.txt U

LAB3-Detecting_Anomalies_with_Prophet > containers > monitor_app > requirements.txt

```
pandas
prophet
prometheus_client
scikit-learn
numpy==1.26.0
requests
```

Added the monitor_app to the prometheus configuration file on port 8003:

LAB3-Detecting_Anomalies_with_Prophet > containers > prometheus > ! config.yml

```
global:
  scrape_interval: "5s"
  evaluation_interval: "5s"

scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets:
          - "node_exporter:9100"
  - job_name: "postgres_exporter"
    static_configs:
      - targets:
          - "postgres_exporter:9187"
  - job_name: "push_gateway"
    static_configs:
      - targets:
          - "push_gateway:9091"
```

```
  - job_name: "app_one"
    static_configs:
      - targets:
          - "app_one:8000"
  - job_name: "monitor_app"
    static_configs:
      - targets:
          - "monitor_app:8003"
```

Also added the monitor_app to the docker compose file on the port 8003

LAB3-Detecting_Anomalies_with_Prophet > docker-compose.yml

```
services:
  app_one:
    container_name: "app_one"
    build: "./containers/app_one"
    ports:
      - "8000:8000"

  app_two:
    container_name: "app_two"
    build: "./containers/app_two"
    ports:
      - "8001:8001"
    environment:
      DOCKER_NETWORK: push_gateway:9091

  prometheus:
    container_name: "prometheus"
    build: "./containers/prometheus"
    ports:
      - "9090:9090"
  grafana:
    container_name: "grafana"
    build: "./containers/grafana"
    ports:
      - "3000:3000"
  node_exporter:
    container_name: "prometheus_node_exporter"
    image: "quay.io/prometheus/node-exporter"
    ports:
      - "9100:9100"
  push_gateway:
    container_name: "prometheus_push_gateway"
    image: "prom/pushgateway"
    ports:
      - "9091:9091"

  postgres:
    container_name: "postgres"
    image: postgres:13.3
    restart: always
    ports:
      - "5432:5432"
    environment:
      POSTGRES_PASSWORD: example
  postgres_exporter:
    container_name: "postgres_exporter"
```

```
47    image: "wrouesnel/postgres_exporter"
48    ports:
49      - "9187:9187"
50    environment:
51      DATA_SOURCE_NAME: "postgresql://postgres:example@postgres:5432/postgres?sslmode=disable"
52    depends_on:
53      - postgres
54  monitor_app:
55    container_name: "monitor_app"
56    build: "./containers/monitor_app"
57    ports:
58      - "8003:8003"
```

Then readjusted the monitor_app for the docker container:

monitor_app.py M

LAB3-Detecting_Anomalies_with_Prophet > containers > monitor_app > monitor_app.py

```python
import pandas as pd
from prophet import Prophet
from prometheus_client import Gauge, start_http_server
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error
import requests
import numpy
from datetime import datetime, timedelta
import logging
import time
import pickle

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logging.getLogger('prophet').setLevel(logging.WARNING)
logging.getLogger('cmdstanpy').setLevel(logging.WARNING)

anomaly_count = Gauge('anomaly_count', 'Number of anomalies detected')
mae_metric = Gauge('mae_metric', 'Mean Absolute Error')
mape_metric = Gauge('mape_metric', 'Mean Absolute Percentage Error')
start_http_server(8003)

def fetch_metric_data(metric_name, start_time, end_time, step='15s'):
    url = f"http://prometheus:9090/api/v1/query_range"
    params = {
        'query': metric_name,
        'start': start_time.timestamp(),
        'end': end_time.timestamp(),
        'step': step
    }
    results = requests.get(url, params=params).json().get('data', {}).get('result', [])
    if not results:
        return pd.DataFrame()
    df = pd.DataFrame(results[0]['values'], columns=['ds', 'y'])
    df['ds'] = pd.to_datetime(df['ds'], unit='s')
    df['y'] = df['y'].astype(float)
    return df

def train_model(train_data):
    model = Prophet(growth='flat', daily_seasonality=False, weekly_seasonality=False, yearly_seasonality=False)
    model.fit(train_data)
    with open('prophet_model.pkl', 'wb') as f:
        pickle.dump(model, f)
    return model
```

```python
def load_model():
    with open('prophet_model.pkl', 'rb') as f:
        model = pickle.load(f)
    return model

def detect_anomalies(model, test_data):
    forecast = model.predict(test_data)
    forecast['ds'] = pd.to_datetime(forecast['ds'])
    merged = forecast.merge(test_data, on='ds', how='inner')
    return merged[merged['y'] > merged['yhat_upper']], forecast

def main():
    all_results = pd.DataFrame()
    cumulative_anomaly_count = 0
    model = None

    # Train the model once
    end_time = datetime.now()
    train_data = fetch_metric_data('train_gauge', end_time - timedelta(minutes=5), end_time)
    if not train_data.empty:
        model = train_model(train_data)
        logging.info("Model trained successfully")
    else:
        logging.error("Training data is empty, unable to train the model.")
        return

    while True:
        try:
            if not model:
                model = load_model()
                logging.info("Loaded saved model successfully")

            # Fetch test data
            test_data = fetch_metric_data('test_gauge', end_time, datetime.now())
            if test_data.empty:
                logging.warning("Test data empty, skipping cycle")
                continue

            # Detect anomalies using the pre-trained model
            anomalies, forecast = detect_anomalies(model, test_data)
            cumulative_anomaly_count += len(anomalies)
            anomaly_count.set(cumulative_anomaly_count)

            # Calculate and set metrics
            mae = mean_absolute_error(test_data['y'], forecast['yhat'][:len(test_data)])
            mape = mean_absolute_percentage_error(test_data['y'], forecast['yhat'][:len(test_data)])
            mae_metric.set(mae)
            mape_metric.set(mape)

            # Log results
            current_results = forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].copy()
            current_results['y'] = test_data['y'].values
            current_results['timestamp'] = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
            current_results['anomaly_count'] = cumulative_anomaly_count
            current_results['ds'] = current_results['ds'].dt.strftime('%Y-%m-%d %H:%M:%S')
            current_results = current_results[['timestamp', 'ds', 'y', 'yhat', 'yhat_lower', 'yhat_upper', 'anomal
            all_results = pd.concat([all_results, current_results], ignore_index=True)

            logging.info(f"Anomalies: {len(anomalies)}, Cumulative: {cumulative_anomaly_count}, MAE: {mae:.4f}, MA
            pd.set_option('display.max_rows', None)
            pd.set_option('display.max_columns', None)
            pd.set_option('display.width', None)
            logging.info("All Results:\n" + all_results.to_string(index=False))
```

```
        except Exception as e:
            logging.error(f"An error occurred: {e}")
        time.sleep(60)

f __name__ == "__main__":
    main()
```

Anomaly count, Mae and Mape metrics added to the application:

```
anomaly_count, mae_metric, mape_metric = Gauge('anomaly_count', 'Number of anomalies detected'),
Gauge('mae_metric', 'Mean Absolute Error'),
Gauge('mape_metric', 'Mean Absolute Percentage Error')
```

```
mae = mean_absolute_error(test_data['y'], forecast['yhat'][:len(test_data)])
mape = mean_absolute_percentage_error(test_data['y'], forecast['yhat'][:len(test_data)])
mae_metric.set(mae)
mape_metric.set(mape)
```

Then restarted the docker compose.

```
oadewusi@oluSec:~/aiops/Assignments/LAB3-Detecting_Anomalies_with_Prophet/containers/monitor_app$ docker compose up
[+] Running 9/0
 ✓ Container prometheus_push_gateway   Created
 ✓ Container monitor_app               Created
 ✓ Container prometheus_node_exporter  Created
 ✓ Container app_two                   Created
 ✓ Container prometheus                Created
 ✓ Container grafana                   Created
 ✓ Container postgres                  Created
 ✓ Container app_one                   Created
 ✓ Container postgres_exporter         Created
```

The console logs indicating monitor_app was deployed successfully.

```
monitor_app          | 2024-09-26 11:22:52,044 - INFO - Chain [1] start processing
monitor_app          | 11:22:52 - cmdstanpy - INFO - Chain [1] done processing
monitor_app          | 2024-09-26 11:22:52,060 - INFO - Chain [1] done processing
monitor_app          | 2024-09-26 11:22:52,065 - INFO - Model trained successfully
monitor_app          | 2024-09-26 11:22:52,109 - INFO - Anomalies: 1, Cumulative: 1, MAE: 0.2313, MAPE: 0.5354
monitor_app          | 2024-09-26 11:22:52,111 - INFO - All Results:
monitor_app          |          timestamp                 ds        y     yhat  yhat_lower  yhat_upper  anomaly_count
monitor_app          | 2024-09-26 11:22:52 2024-09-26 11:22:51 0.431974 0.200705    0.045079    0.365309              1
grafana              | logger=infra.usagestats t=2024-09-26T11:23:28.145465939Z level=info msg="Usage stats are ready to report"
monitor_app          | 2024-09-26 11:23:52,207 - INFO - Anomalies: 4, Cumulative: 5, MAE: 0.3767, MAPE: 0.5750
monitor_app          | 2024-09-26 11:23:52,209 - INFO - All Results:
monitor_app          |          timestamp                 ds        y     yhat  yhat_lower  yhat_upper  anomaly_count
monitor_app          | 2024-09-26 11:22:52 2024-09-26 11:22:51 0.431974 0.200705    0.045079    0.365309              1
monitor_app          | 2024-09-26 11:23:52 2024-09-26 11:22:51 0.431974 0.200705    0.041852    0.360859              5
monitor_app          | 2024-09-26 11:23:52 2024-09-26 11:23:06 0.163568 0.200705    0.042954    0.360938              5
monitor_app          | 2024-09-26 11:23:52 2024-09-26 11:23:21 0.804012 0.200705    0.041042    0.353590              5
monitor_app          | 2024-09-26 11:23:52 2024-09-26 11:23:36 0.939959 0.200705    0.045730    0.358131              5
monitor_app          | 2024-09-26 11:23:52 2024-09-26 11:23:51 0.473158 0.200705    0.028971    0.370162              5
```

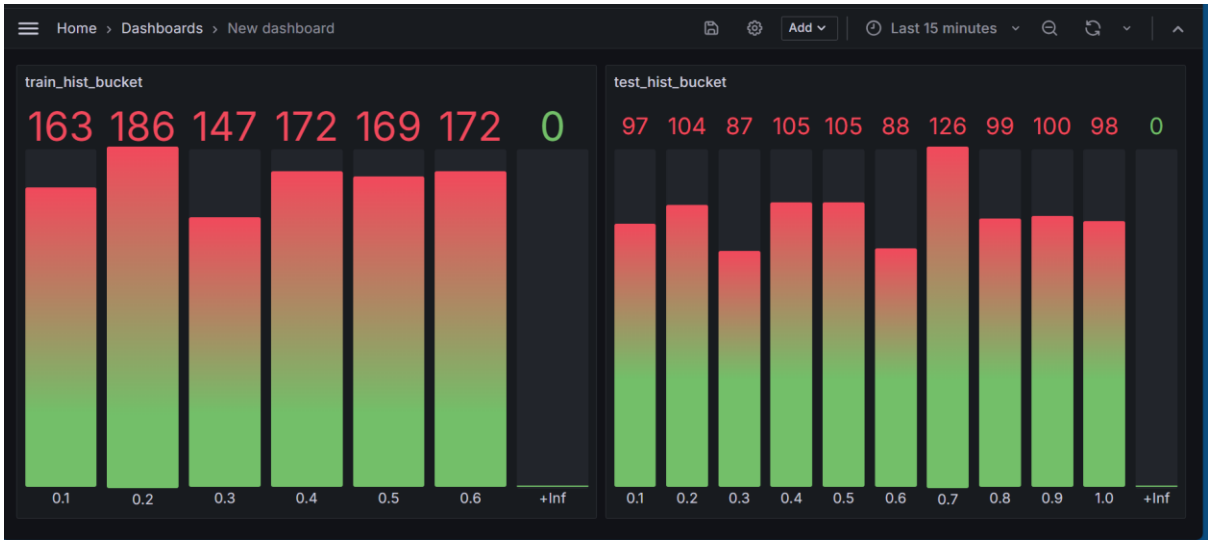
Continuous training of the model

```
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:23:36 0.939959 0.200705    0.043059    0.359490             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:23:51 0.473158 0.200705    0.043954    0.362370             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:24:06 0.710206 0.200705    0.043304    0.363511             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:24:21 0.569561 0.200705    0.040475    0.378626             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:24:36 0.046055 0.200705    0.041441    0.371167             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:24:51 0.859211 0.200705    0.040704    0.380735             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:25:06 0.912549 0.200705    0.031027    0.361900             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:25:21 0.751172 0.200705    0.045111    0.360394             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:25:36 0.166525 0.200705    0.044064    0.364680             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:25:51 0.914562 0.200705    0.053644    0.354454             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:26:06 0.620237 0.200705    0.034509    0.365141             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:26:21 0.752680 0.200705    0.039393    0.373474             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:26:36 0.501231 0.200705    0.041129    0.357132             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:26:51 0.041204 0.200705    0.025338    0.357666             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:27:06 0.759906 0.200705    0.039015    0.369643             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:27:21 0.398706 0.200705    0.037144    0.377350             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:27:36 0.718470 0.200705    0.039478    0.363589             52
monitor_app          | 2024-09-26 11:27:52 2024-09-26 11:27:51 0.753316 0.200705    0.058846    0.370417             52
```

Legend:

timestamp: This column represents the time at which the result was generated or recorded.
ds: This column stands for "datestamp" and represents the time of the observed data points in the test dataset.
y: This column contains the actual observed values of the metric from the test data.
yhat: This is the forecasted value for the metric generated by the Prophet model. It's the predicted value based on the trained model, for the given ds time point.
yhat_lower: This represents the lower bound of the predicted confidence interval.
yhat_upper: This represents the upper bound of the predicted confidence interval.
anomaly_count: This column keeps track of the cumulative number of anomalies detected up to that point in time.

Histograms of my request_time metrics using **train_hist_bucket** and **test_hist_bucket**



Grafana dashboard of the **train_guage**, **test_guage** , **mape_metric**, **anomaly_count** and **mae_metric** in **5 minutes** timeframe

Grafana dashboard of the **train_guage**, **test_guage** , **mape_metric**, **anomaly_count** and **mae_metric** in **10 minutes** timeframe

Grafana dashboard of the **train_guage**, **test_guage** , **mape_metric**, **anomaly_count** and **mae_metric** in **15 minutes** timeframe

Grafana dashboard of the **train_hist_bucket, test_hist_bucket train_guage**, **test_guage** , **mape_metric**, **anomaly_count** and **mae_metric** in **15 minutes** timeframe

Lab Task – Answer the following questions.

1. Reasonable Baseline of Data for Prophet Model

A reasonable baseline of data for using a Prophet model in a production system depends on the frequency and nature of the data. For time series forecasting, it's recommended to have at least one year of historical data to capture seasonal patterns and trends effectively. However, if the data is more granular (e.g., minute-level data), a few months of data might suffice.

(a) Considerations:

- Prophet is designed to handle seasonality and trends. Having at least one full cycle of seasonal data (e.g., one year for yearly seasonality) helps the model learn these patterns.
- For high-frequency data (e.g., minute-level), a few months might be enough, but for daily or weekly data, a longer history is beneficial.
- Ensuring the data is clean and consistent. Missing values and outliers should be handled appropriately.

(b) Operational Challenges:

- Large datasets can lead to increased computational time and resource usage. This might require more powerful hardware or cloud resources.
- Depending on the data size and frequency, training the model can take significant time. This might delay the deployment of the model or updates.
- Storing large volumes of historical data can be challenging and might require efficient data storage solutions.

2. Continuous Retraining vs. Manual Review/Approval

Continuous Retraining:

Advantage:

- The model can quickly adapt to new patterns and changes in the data, improving its accuracy over time.
- Reduces the need for manual intervention, saving time and resources.

Disadvantage:

- Continuous retraining without proper validation can lead to overfitting, where the model performs well on training data but poorly on unseen data.
- Frequent retraining can be computationally expensive and resource intensive.
- The model might learn from anomalies or noise in the data, leading to inaccurate predictions.

Manual Review/Approval:

Advantage:

Quality Control: Allows for human oversight to ensure the model is performing as expected and not learning from anomalies or noise.

Stability: Reduces the risk of overfitting and ensures the model is only updated when necessary.

Disadvantage:

- Manual review can introduce delays in updating the model, potentially reducing its adaptability to new patterns.

- Requires human resources for review and approval, which can be time-consuming and costly.

Disadvantage of Fully Automatic Operation:

- Without manual review, there is a risk of the model learning from incorrect or anomalous data, leading to poor performance.
- Continuous retraining without proper validation can cause the model to overfit to recent data, reducing its generalizability.
- Frequent retraining can be resource-intensive, requiring significant computational power and storage.

Recommendations

- Implementing a hybrid approach where the model retrains periodically (e.g., weekly, or monthly) with manual review and approval for significant updates.
- Continuously monitoring the model's performance using metrics like MAE and MAPE. Setting up alerts for significant deviations.
- Using a robust validation strategy to ensure the model is not overfitting and performs well on unseen data.
- Ensuring efficient data storage and management practices to handle large volumes of historical data.